Works Cited Document - “Definitely Spaceship Pilot Certified”
Jack P, Jack G, Abbi W

Jack Pelletier

When creating the asteroids, we had to make a seemingly infinite amount of 3d objects. I asked ChatGPT and Google what would be a good way to build this would be to use ThreeJS’s InstancedMesh function to render these objects in a single call. I also asked ChatGPT for help on generating the different types of drifts mentioned in the document, asking it “how should I implement these different drift types into the asteroids”

```
// Starting position
let x = asteroid.x;
let y = asteroid.y;


// Movement type 0: Side-to-side drift
if (asteroid.movementType === 0) {

    x += Math.sin(time * asteroid.driftSpeed + asteroid.phase) * asteroid.driftAmount;

}


// Movement type 1: Parabolic drift
else if (asteroid.movementType === 1) {

    const t = Math.sin(time * asteroid.driftSpeed + asteroid.phase);

    x += t * asteroid.driftAmount;

    y += (t * t - 0.5) * asteroid.driftAmount;

}


// Movement type 2: Circular drift
else {

    x += Math.cos(time * asteroid.driftSpeed + asteroid.phase) * asteroid.driftAmount;

    y += Math.sin(time * asteroid.driftSpeed + asteroid.phase) * asteroid.driftAmount;

}
```

I also asked for help on the GLSL shader making the patterns on the asteroids unique from one another, and it wasn’t just the exact same texture over and over. I asked ChatGPT “How can we make it so the patterns and colors are randomly applied to each asteroid generated in the space”

```
    void main() {

        // Give each asteroid unique texture coordinates
        vec2 uniqueUv = vUv + vec2(
            vTextureSeed * 13.7,
            vTextureSeed * 29.3
        );

        // Animate each asteroid's texture
        vec2 animatedUv = uniqueUv + vec2(uTime * 0.01, uTime * 0.005);

        // Generate different surface patterns
        float pattern1 = noise(animatedUv * 12.0);
        float pattern2 = noise(animatedUv * 30.0);

        float rockyPattern = pattern1 * 0.7 + pattern2 * 0.3;

        // Create different shades of each asteroid's color
        vec3 darkColor = vColor * 0.35;
        vec3 lightColor = vColor * 1.5;

        vec3 finalColor = mix(darkColor, lightColor, rockyPattern);

        // Calculate directional lighting
        vec3 lightDirection = normalize(vec3(1.0, 1.0, 1.0));

        float lighting = max(dot(normalize(vNormal), lightDirection), 0.0);

        finalColor *= 0.3 + lighting * 0.7;

        gl_FragColor = vec4(finalColor, 1.0);

    }
`,
```

Jack Griffin

I used Google Gemini to help position and get the overall shape of the ship, and from there I added the details and fine-tuned positions and rotations of the geometries. I also needed assistance in getting the formulas for the animated textures of the screens to work, getting vertex and fragment shaders to work correctly, and adding in the UI to display controls and make them stand out. Some of my prompts included: *“How can we use rasterization to create animated screens for the ship’s dashboard?”* and *“How can we add a boost feature that increases ship speed and FOV when holding down LMB?”*.

```
    this.buildCockpit();
  }

  buildCockpit() {
    this.buildFloorAndPedals();
    this.buildDashboard();
    this.buildEnclosedWallsAndRoof();
    this.buildSeat();
    this.buildViewportFrame();
    this.buildFlightControls();
  }

  // floor and pedals
  buildFloorAndPedals() {
    const floorGroup = new THREE.Group();

    // main floor
    const floor = new THREE.Mesh(
      new THREE.BoxGeometry(2.0, 0.1, 2.0),
      this.materials.hull
    );
    floor.position.set(0, -1, -0.2);
    floorGroup.add(floor);

    // foot ramp
    const ramp = new THREE.Mesh(
      new THREE.BoxGeometry(1.5, 0.05, 0.5),
      this.materials.hull
    );
    ramp.position.set(0, -0.8, -1.05);
    ramp.rotation.x = Math.PI * 0.2;
    floorGroup.add(ramp);

    // foot pedals
    const pedalGeo = new THREE.BoxGeometry(0.2, 0.3, 0.05);

    const leftPedal = new THREE.Mesh(pedalGeo, this.materials.accent);
```

```
// glsl vertex shader
const screenVertexShader = `
  varying vec2 vUv;
  varying vec3 vNormal;

  void main() {
    vUv = uv;
    vNormal = normalize(normalMatrix * normal);
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
  }
`;

// glsl fragment shader
const screenFragmentShader = `
  uniform float uTime;
  uniform vec3 uColor;
  varying vec2 vUv;

  void main() {
    // Procedural grid lines
    vec2 gridUv = fract(vUv * 10.0);
    float lineX = step(0.90, gridUv.x) + step(gridUv.x, 0.10);
    float lineY = step(0.90, gridUv.y) + step(gridUv.y, 0.10);
    float gridPattern = clamp(lineX + lineY, 0.0, 1.0);

    // Animated vertical scanlines
    float scanline = sin(vUv.y * 40.0 - uTime * 4.0) * 0.2 + 0.8;

    // Pulse wave
    float pulse = sin(uTime * 2.0 + vUv.x * 4.0) * 0.15 + 0.85;

    vec3 finalColor = uColor * (gridPattern * 0.7 + 0.3) * scanline * pulse;

    // Explicit Alpha Blending factor
    gl_FragColor = vec4(finalColor, 0.85);
  }
`;
```

Abbigail Weber

To begin the collision detection I followed a tutorial from the website “SpicyYoghurt” to try and get the base of the code for creating collision detection. The implementation of this code allowed me to get to the point where I was able to recognize collisions, using a console log output to confirm this, however I was not able to get the physics of the asteroid bouncing off the ship to function properly. I used ChatGpt to debug the collision detection code as it had some errors, as well as modifying the code to be more optimized and condensed. A particular error that I struggled with was that some of the asteroids would bounce off as intended, while others were seemingly still clipping through the ship. I asked ChatGpt “Why are some of my asteroids responding correctly while others seem to phase through?” and I was able to find an error in the asteroid file where the position of the asteroid was being updated in two different functions, causing it to not always respond as intended.

```
if (asteroid.movementType === 0) {
    driftX = Math.sin(phase) * asteroid.driftAmount;
} else if (asteroid.movementType === 1) {
    driftX = Math.sin(phase) * asteroid.driftAmount;
} else if (asteroid.movementType === 2) {
    driftX = Math.cos(phase) * asteroid.driftAmount;
}

    Let x = asteroid.x + driftX;
    Let y = asteroid.y;


    if (asteroid.movementType === 1) {
        const t = Math.sin(phase);
        y += (t * t - 0.5) * asteroid.driftAmount;
    }else if (asteroid.movementType === 2) {
    y += Math.sin(phase) * asteroid.driftAmount;
    }

    asteroidPosition.set(x, y, asteroid.z);

    spaceshipBox.clampPoint(asteroidPosition, closestPoint);

    collisionNormal.subVectors(asteroidPosition, closestPoint);

    const distance = collisionNormal.length();

    const scale =
    asteroid.size *(1 + 0.2 * Math.sin(time * asteroid.driftSpeed + asteroid.phase));

    const radius = scale;

    if (distance < radius + 0.3) {
        asteroid.isColliding = true;
        console.log("collision detected")//checks to make sure collsion detection is working
    if (distance < 0.0001){
    const distances = [
       { value: asteroidPosition.x - spaceshipBox.min.x, axis: 0, sign: -1 },
        { value: spaceshipBox.max.x - asteroidPosition.x, axis: 0, sign: 1 },
        { value: asteroidPosition.y - spaceshipBox.min.y, axis: 1, sign: -1 },
        { value: spaceshipBox.max.y - asteroidPosition.y, axis: 1, sign: 1 },
        { value: asteroidPosition.z - spaceshipBox.min.z, axis: 2, sign: -1 },
        { value: spaceshipBox.max.z - asteroidPosition.z, axis: 2, sign: 1 }
    ];
```